VMCTL — Technical Whitepaper & Roadmap

vm-bhyve front-ends: Desktop (Slint) + Terminal (ratatui) over one shared core

Version 0.1 · October 2026 · Status: foundation landed, design locked

1. Vision & Design Principles

VMCTL is a pair of front-ends for **vm-bhyve** (FreeBSD's bhyve manager) that make virtualization **powerful and easy** — the VirtualBox experience, on a real hypervisor. Two front-ends, one core: anything you do in the Desktop app shows up in the Terminal app and vice-versa, because both drive the same **vmctl-core** state and the same **vm(8)** commands.

> **One core, two front-ends.** Desktop (Slint) and Terminal (ratatui) are thin shells over a shared Rust core. No duplicated logic, no drift.

> **Honest by construction.** Every action maps to a real **vm(8)** subcommand. Exit codes and stderr are surfaced verbatim — never swallowed, never faked.

> **Safe by default.** Plan-before-run for anything destructive. Root gate enforced. Lock semantics respected (a machine locked by another host is never touched without explicit confirmation).

> **Plain-English assistant.** Natural language → intent → validated plan → confirm → execute. The assistant proposes; the human confirms; the core executes.

> **Grounded in the real tool.** The design system, feature IDs, and command surface are taken from the working prototype (*vmctl-prototype.tsx*) and the live vm-bhyve 1.7.5 install — not invented.

2. Architecture

The system is layered so that every capability lives in exactly one place and both front-ends reuse it.

Layer	Responsibility	Notes
vmctl-core	Typed **vm(8)** wrapper + VM model + actions + state machine + assistant engine. Honest exit codes.	Scaffolded: *vmbhyve.rs*, *actions.rs*
Backend trait	Abstract VM ops (list/create/start/stop/…). vm-bhyve is the first implementation.	Future: VirtualBox, QEMU, cloud
Assistant engine	Plain-English → intent → validated action plan → confirm → execute. Sequential runner stops at first non-zero exit and keeps real stderr.	Shared by TUI + Desktop
Desktop (Slint)	Windowed app: machines, snapshots, switches/storage, activity, assistant drawer, creation wizard.	Mission Control palette
Terminal (ratatui)	Panes: VM list, action bar, wizard, config editor, console attach, log tail.	ca0e750: list + actions done

2.1 Shared core contract

Both front-ends call the same core API. The core owns: (a) the typed wrapper that shells out to **vm(8)** and parses its tabular output (including multi-word states like *Locked (nas01)* and *Running (4182)*); (b) the VM model and action layer (Start/Stop/Restart/Suspend/Resume/PowerOff/Snapshot/Rollback/Clone/Destroy/Create/Install/Console/Unlock); (c) the assistant planner that turns a sentence into a validated plan with risk/touches/memory-after metadata and per-step undo; (d) the sequential plan executor that stops at the first failure and preserves the real stderr.

3. Design System

The **Mission Control** palette: crimson is action, ember is focus/assistant, and state colours never reuse crimson. Type stack: Orbitron (display), Rajdhani (UI), JetBrains Mono (data/commands).

Token	Value	Role
crimson	`#DC2F45`	Primary action, destructive emphasis
ember	`#FF7A3D`	Focus, assistant, suggested next step
green	`#34D399`	Running / success
amber	`#F5B53D`	Suspended / warning
violet	`#B79CFF`	Locked (held by another host)
bg / panel / surface	`#0A0B0D / #0F1114 / #14171B`	Surfaces

4. Feature Specification

4.1 Assistant (F-401 … F-405)

F-401 Intent parsing — plain English → intent (create/start/stop/snapshot/clone/explain/…).

F-402 Validated plan — every plan shows steps, exact **vm(8)** commands, risk, what it touches, memory-after, and per-step undo.

F-403 Suggested next step — each machine surfaces a context-aware default action (Running → Open console; Stopped → Start; Suspended → Resume; Locked → Explain lock).

F-404 Sequential execution — run the plan step-by-step; stop at the first non-zero exit; keep the real stderr; nothing after the failure runs.

F-405 Explain errors — surface the actual vm(8) stderr and explain the cause (e.g. a run.lock held by another host) without touching anything.

4.2 VM lifecycle & inventory

List (live **vm list**), create (template → cpu/memory/disk/loader/ISO), install, start, stop, restart, suspend, resume, poweroff, destroy, clone. Templates: Ubuntu, FreeBSD, Debian, Windows, OpenBSD, Alpine — each with sensible defaults for cpu/mem/disk/loader/ISO.

4.3 Snapshots & images

Snapshot (with **-f** for a live/crash-consistent snapshot of a running guest), rollback (stopped only), clone-from-snapshot, and snapshot listing with size/when. The assistant flags that a **-f** snapshot of a running disk is like a power cut — fine for rollback, and offers a quiesce-first alternative.

4.4 Network & storage

Switches (standard / standard-nat / private) with plain-English descriptions; add NIC to switch; add disk (zvol) to a machine; datastores (ZFS datasets) with used/free. The assistant explains effects in guest terms (e.g. "it appears as vtnet1 after a restart").

4.5 Safety model

Root gate — vm(8) needs root; without it the app is read-only and says so, with the exact command to retry (*doas vmctl-desktop*).

Lock semantics — a machine with a run.lock held by another host is never started/unlocked without explicit human confirmation that the other host is gone.

Plan-before-run — destructive or multi-step actions always show the plan and wait for confirmation.

Honest exit codes — every command's real exit code and stderr are shown; failures stop the plan.

5. Roadmap

Milestone	Scope	Exit criteria	Status
M0 Foundation	Crate + typed vm(8) wrapper + VM list + action layer.	Compiles; 6 tests green; live **vm list** parsed (testvm).	**Done** — 0c18a73, ca0e750
M1 Core + Assistant	Full action layer, assistant planner + sequential runner, plan cards.	planFor covers create/start/stop/snapshot/explain; runner stops at first failure with real stderr.	Next

Milestone	Scope	Exit criteria	Status
M2 Desktop shell	Slint app: machines/snapshots/network/activity views + assistant drawer + wizard.	Parity with prototype; anything done here shows in TUI.	Planned
M3 TUI panes	ratatui: list + actions + wizard + config editor + console attach + log tail.	Feature parity with Desktop for core flows.	Planned
M4 Polish & hardening	Error UX, lock flows, docs, packaging, cross-host awareness.	Safe-by-default verified; docs published.	Planned

6. Risks & Mitigations

vm(8) output drift — parsing is pinned to vm-bhyve 1.7.5 and covered by unit tests (including multi-word states); a version bump is a test change, not a silent break.

Root requirement — surfaced honestly as a read-only mode with the exact retry command, never a crash.

Destructive actions — plan-before-run + per-step undo + stop-at-first-failure; locks respected.

Two front-ends drifting — impossible by construction: both are thin shells over one core.

Generated from the working prototype (*vmctl-prototype.tsx*) and the live vm-bhyve 1.7.5 command surface. Foundation commits: 0c18a73 (crate + wrapper), ca0e750 (VM-list TUI + action layer).